

# QuickDelivery — Arquitetura

**Aluno:** Joaquim Vilela

**Sprint:** 1 – Arquitetura e Backend REST

```
flowchart LR
    subgraph Mobile["Aplicações Móveis (Flutter)"]
        CustomerApp["Customer App"]
        DeliverymanApp["Deliveryman App"]
    end

    subgraph Backend["Backend"]
        API["REST API<br/>Node.js + Express + TypeScript"]
        Auth["Auth Middleware<br/>JWT + Roles"]
        Domain["Services<br/>Regras de negócio"]
    end

    subgraph Messaging["Mensageria - Sprint 2"]
        MOM["RabbitMQ<br/>(planejado)"]
    end

    subgraph Storage["Persistência"]
        DB["PostgreSQL 16<br/>Docker"]
    end

    CustomerApp -- "HTTPS / REST (JSON)<br/>Bearer Token" --> API
    DeliverymanApp -- "HTTPS / REST (JSON)<br/>Bearer Token" --> API
    API --> Auth
    Auth --> Domain
    Domain -- "TCP / SQL via Prisma" --> DB
    Domain -. "AMQP 0-9-1 (publish)<br/>Sprint 2" .-> MOM
    MOM -. "AMQP 0-9-1 (consume)<br/>Sprint 2" .-> DeliverymanApp
    MOM -. "AMQP 0-9-1 (consume)<br/>Sprint 2" .-> CustomerApp
```

A arquitetura do QuickDelivery é distribuída e preparada para EDA. Na Sprint 1, os clientes da API consomem endpoints REST em Node.js + Express + TypeScript com payloads JSON e autenticação por Bearer Token. O middleware de autenticação valida o token JWT e disponibiliza `userId` e `role` para as regras de negócio, permitindo separar permissões de `CUSTOMER` e `DELIVERYMAN`.

A API persiste dados no PostgreSQL através do Prisma ORM. O modelo atual usa `users`, `deliveries` e `audit_logs`: clientes criam entregas próprias, entregadores visualizam entregas pendentes ou atribuídas a eles, e as transições de status são validadas nos services.

O RabbitMQ aparece no diagrama como componente planejado para a Sprint 2. A partir dessa etapa, eventos como `delivery.created`, `delivery.accepted` e `delivery.status_changed` serão publicados via AMQP e consumidos pelos componentes interessados, reduzindo polling e desacoplando produtores e consumidores.